



Práctica 5 Programación defensiva, serialización y documentación con JavaDoc.

Objetivos de aprendizaje

1. Validar el código usando programación defensiva.	Saber validar el código mediante la aplicación de técnicas de programación defensiva (fail fast), anticipando y gestionando errores a través de excepciones y validaciones para garantizar la robustez del programa ante entradas no válidas o inesperadas.
2. Serializar y deserializar objetos para almacenar y recuperar datos	Saber cómo guardar y recuperar el estado de objetos complejos usando las herramientas de serialización proporcionadas por Java.
3. Documentar el código fuente utilizando JavaDoc.	Documentar el código fuente con JavaDoc aplicando los principios del diseño por contrato, expresando precondiciones, postcondiciones e invariantes de clase para mejorar la claridad, la fiabilidad y el mantenimiento del software..
4. Convenciones de codificación en Java	Saber aplicar adecuadamente las normas de estilo y convenciones de codificación de Java (nombres de variables, formato, comentarios, indentación, etc.), mostrando código claro y legible.

Descripción

En esta última práctica guiada se aplican técnicas de programación defensiva basadas en el principio fail-fast, orientadas a detectar y gestionar los errores tan pronto como se producen, antes de que afecten al flujo general del programa. Asimismo, se implementará la serialización para almacenar en memoria secundaria los objetos que gestionan los recursos del sistema, garantizando la persistencia del estado. Finalmente, se elaborará la documentación del proyecto mediante la herramienta JavaDoc, describiendo de forma estructurada las clases, métodos y contratos del programa.

1. Primera parte. Validación del código.

1.1. Validación de la selección de opciones

Modificar el método `readInt` para que capture y trate una excepción del tipo `NumberFormatException`, en el caso de que la cadena de caracteres (String) que se introduzca no se corresponda con un número entero, de tal forma que si se lanza esta excepción se le indique al usuario que debe introducir un número válido y se vuelva a leer la entrada.

1.2. Validación del formato de la respuesta del usuario.

Cada juego validará su entrada justo en el punto donde la consume, en este caso, dentro del método **juega**, apoyándose en una clase Validaciones (paquete funcionalidades) accesible desde cualquier clase del proyecto. Si la entrada no cumple el formato esperado, el método lanzará una excepción específica. Esta forma de trabajo obligará a modificar la cabecera del método **juega** en el interfaz Jugable, de modo que indique expresamente que puede arrojar excepciones.

Los métodos de validación se diseñarán de modo que, si el valor es correcto, retornen el mismo valor validado, lo que permitirá encadenar su uso de forma directa en el flujo del código.

JuegoAdivinaNumero. Antes de procesar la respuesta del usuario en el cuerpo de **juega**, se verificará que la cadena no está vacía y que representa un número entero (todos sus caracteres son dígitos). Para ello se crearán dos métodos estáticos en Validaciones:

- **cadenaNoVacía:** Recibe una cadena “s” y la devuelve validada si no está vacía ni es null; de lo contrario lanza `IllegalArgumentException` si está vacía o si es null, con mensajes claro del error.
- **numeroEntero:** Recibe una cadena “s” y devuelve el número entero correspondiente si la cadena tiene un formato válido. En caso contrario, lanza una `NumberFormatException` con un mensaje que describa la causa del error. Este método sustituirá al uso de **Integer.parseInt**, empleado hasta ahora, para validar y obtener un número entero a partir de una cadena.

JuegoAhorcadoIngles. Antes de procesar la entrada en el método **juega**, se deberá comprobar que la cadena introducida no está vacía y que corresponde a una única letra minúscula. Para ello se emplearán dos métodos estáticos de la clase Validaciones: primero se invocará **cadenaNoVacía** para asegurar que la entrada no está vacía, y a continuación un nuevo método estático denominado **letraMinuscula** para verificar que se trata de una única letra minúscula. Este último método recibe la cadena y devuelve un carácter validado si se cumple el formato o lanzará una excepción de tipo `JuegoAhorcadoInglesExcepcion` (paquete excepciones). Esta clase deberá ser creada por el estudiante extendiendo `Exception`, con un constructor que reciba una cadena descriptiva del error.

Este enfoque obliga a manejar las excepciones en la capa de interacción (método **jugar** de la clase `Menu`). La lectura de la respuesta se ubicará en un bucle protegido con una estructura `try/catch`: mientras se produzca una excepción de validación, se informará del error y se volverá a solicitar la entrada; solo cuando la entrada sea válida se continuará con la jugada.

1.3. Validación de la entrada del diccionario mediante excepciones.

La validación de la palabra seleccionada para el juego del ahorcado se realiza también mediante el lanzamiento de excepciones. En este caso, la comprobación se llevará a cabo durante la fase de inicialización del juego, dentro del método **reiniciaPartida()** de la clase **JuegoAhorcadoIngles**, asegurando que ninguna palabra del diccionario que contenga caracteres no válidos sea utilizada en la partida.

Método de validación.

La validación se implementará en un nuevo método estático denominado **chequeaPalabra**, situado en la clase Validaciones del paquete funcionalidades. Este

método recibe como parámetro una cadena de texto (String) que representa la palabra seleccionada aleatoriamente del diccionario. Su misión es comprobar que todos los caracteres de dicha palabra son letras minúsculas. Si detecta algún carácter que no cumpla esta condición (por ejemplo, números, mayúsculas o símbolos), lanzará una excepción de tipo `JuegoAhorcadoInglesExcepcion` (ubicada en el paquete `excepciones`) con un mensaje descriptivo que indique la causa del error. En caso contrario, el método devolverá la cadena validada sin modificarla, permitiendo continuar la ejecución del juego.

Modificación del método `reiniciaPartida` en `JuegoAhorcadoIngles`.

El método **`reiniciaPartida()`** deberá modificarse para integrar la validación de la palabra seleccionada. Para ello:

1. Tras obtener una palabra aleatoria del diccionario, se invocará el método **`chequeaPalabra`** dentro de un bloque `try`.
2. Si el método no lanza excepción, la palabra se considera válida y se utiliza para inicializar los `ArrayList` que contienen la palabra a adivinar y los guiones.
3. Si se produce una excepción `JuegoAhorcadoInglesExcepcion`, el bloque `catch` deberá capturarla, mostrar el mensaje de error y volver a extraer una nueva palabra aleatoria del diccionario.
4. Este proceso se repetirá dentro de un bucle hasta obtener una palabra válida, garantizando que la partida solo se inicia con palabras que cumplen las reglas establecidas.

Prueba y comprobación.

Una vez implementadas estas modificaciones, se deberá compilar y ejecutar el programa utilizando diccionarios que contengan palabras válidas y otras que incluyan caracteres no permitidos (por ejemplo, con números o mayúsculas). De este modo se comprobará que el método **`chequeaPalabra`** detecta correctamente las palabras erróneas, lanza la excepción correspondiente y que el método **`reiniciaPartida()`** reintenta la selección hasta encontrar una palabra válida.

Nota: Para la implementación de los métodos de la clase `Validaciones`, puedes seguir el esquema propuesto en la clase de funcionalidades para validación utilizada como ejemplo en el *Tema 4: Diseño por Contrato y Programación Defensiva*.

2. Segunda parte. Serialización.

- En esta parte de la práctica se emplearán los dos siguientes métodos de la clase `MyInput` (incluidos en su código):

```
public static <A> void serialize(A a, String nombreFichero) {
    System.out.println("Serializando...");
    try {
        FileOutputStream fos = new FileOutputStream(nombreFichero) ;
        ObjectOutputStream oos = new ObjectOutputStream(fos) ;
        oos.writeObject(a) ;
    } catch (Exception e) {
        System.err.println("Problem: "+e) ;
    }
}
```

```

public static <A> A deserialize(String nombreFichero) {
    System.out.println("DeSerializing...");
    try {
        FileInputStream fis = new FileInputStream(nombreFichero) ;
        ObjectInputStream iis = new ObjectInputStream(fis) ;
        return (A) iis.readObject() ;
    } catch (Exception e) {
        System.err.println("Problem: "+e) ;
    }
    return null ;
}

```

- Modificar las clases JuegoConVidas y JuegosReunidos para que implementen el interfaz Serializable.
- Modificar la clase principal con el siguiente método **main**:

```

public static void main(String[] args) {
    JuegosReunidos jr=MyInput.deserialize("juegos.dat");
    if (jr==null) {
        jr = new JuegosReunidos();
        jr.agregar(new JuegoAdivinaNumero(5));
        jr.agregar(new JuegoAdivinaNumeroPar(5));
        jr.agregar(new JuegoAdivinaNumeroImpar(5));
        jr.agregar(new JuegoAhorcadoIngles(5,MyInput.leeFichero("diccionario.txt")));
    }
    Menu mp=new Menu(jr);
    mp.ejecuta();
    MyInput.serialize(jr,"juegos.dat");
}

```

3. Tercera parte. Documentación

Documentar la parte pública de las clases implementadas utilizando la herramienta **JavaDoc**, empleando para ello todas las funcionalidades que ofrece el IDE. La documentación debe elaborarse siguiendo las directrices del diseño por contrato, de modo que cada elemento público del código (clases, constructores y métodos) exprese claramente sus compromisos con el resto del sistema.

La documentación generada con JavaDoc debe reflejar estos compromisos utilizando las etiquetas apropiadas y, cuando sea necesario, expresando las condiciones mediante fragmentos de código o pseudocódigo.

4. Resultados Evaluación y retroalimentación

4.1. Rúbrica de resultados de aprendizaje

Criterio	Consigue el resultado	No consigue el resultado
1. Validación del código usando programación defensiva	Aplica técnicas de programación defensiva gestionando errores y entradas no válidas mediante excepciones y validaciones. Ejemplifica y explica estos aspectos en el informe.	No valida el código correctamente o no gestiona adecuadamente errores y entradas no válidas. No explica ni ejemplifica cómo lo ha hecho.
2. Serialización y deserialización de objetos	Implementa correctamente la serialización y deserialización para guardar y recuperar el estado de los juegos, y explica claramente su función y relevancia en el programa.	No implementa correctamente la serialización/deserialización, o no comprende su función y relevancia en el sistema.
3.Documentación del código fuente utilizando JavaDoc	Documenta el código fuente de forma clara y completa utilizando JavaDoc, siguiendo las convenciones y buenas prácticas, e incorporando precondiciones, postcondiciones e invariantes según el diseño por contrato.	No documenta adecuadamente el código, o lo hace de manera incompleta, confusa o sin reflejar las condiciones del diseño por contrato ni las convenciones de JavaDoc
4. Convenciones de codificación en Java	El código cumple las normas de estilo y buenas prácticas de Java (nombres, formato, comentarios, indentación).	El código no respeta las normas de estilo y buenas prácticas de Java.

4.2. Preguntas de autoevaluación

Responde a las siguientes preguntas, justificando tus respuestas con fragmentos de código o ejemplos concretos desarrollados en la práctica:

1. ¿En qué partes de tu código se aplica el principio *fail-fast* y cómo ayuda a mantener la integridad del programa?
2. ¿Aplicarías la estrategia *fail fast* en otras partes del código donde inicialmente se usaron validaciones de salida, o finales, mediante booleanos o simplemente no

se han considerado?. Identifica al menos un caso concreto y explica por qué sería beneficioso aplicar el enfoque *fail fast*.

3. ¿De qué manera has serializado y deserializado objetos para guardar y recuperar el estado de los juegos?. Describe donde lo haces y los pasos principales que has seguido. Comenta por qué es importante la serialización en el contexto de tu programa.
4. Reflexiona sobre cómo la documentación que has generado con Javadoc, basada en el diseño por contrato, contribuye a la claridad, reutilización y mantenimiento del código.

4.3. Obteniendo retroalimentación de la IAG

A continuación, deberás solicitar a la IAG con la que trabajes una primera evaluación (retroalimentación) sobre la corrección de tu código y las respuestas que has dado a las preguntas planteadas. Para ello elabora un prompt lo más detallado posible que permita a la IAG realizar una evaluación de la corrección tanto del código como de las respuestas proporcionadas. Es importante que en tu prompt incluyas:

- El enunciado de la práctica
- Los objetivos de aprendizaje
- La rúbrica de logro de resultados de aprendizaje.
- El código desarrollado
- Las preguntas de autoevaluación junto con tus respuestas

La valoración recibida **no debe ser una nota numérica**, sino un análisis en el que se indiquen:

- Qué aspectos están bien resueltos.
- Qué errores o carencias se detectan.
- Qué aspectos son mejorables.
- Si se han alcanzado los objetivos y resultados de aprendizaje previstos, tanto en el código como en las respuestas teóricas.

Es fundamental que tu prompt recoja con claridad todos los aspectos mencionados, de modo que la IAG oriente su análisis exactamente a lo que se espera en la evaluación. Si la respuesta de la IAG no es suficientemente clara, ajustada o útil, mejora tu prompt añadiendo explicaciones adicionales, ejemplos concretos o aclaraciones sobre lo que necesitas.

Recuerda: **la calidad de la retroalimentación recibida depende de la precisión y el contexto que incluyas en tu consulta**. Compara siempre la respuesta de la IAG con los materiales de clase u otras fuentes fiables, o consulta con el/la profesor/a en caso de dudas persistentes. Esto te ayudará a desarrollar un uso crítico y responsable de la IAG.

5. Informe que debe subirse al campus virtual de la asignatura

Una vez obtengas una respuesta satisfactoria, sube a la tarea correspondiente en Moodle un informe en PDF, **utilizando para ello la plantilla que está disponible en el campus virtual**. Este informe incluye:

- Nombre y apellidos
- La IAG utilizada, su versión (si procede) y la fecha.
- El desarrollo de la práctica: El código de la práctica y las respuestas a las preguntas que se formulan en esta práctica.

- Interacción con la IAG: Transcribe el diálogo íntegro y ordenado (tus prompts y las respuestas de la IAG).
- Una valoración personal sobre la utilidad y limitaciones de la retroalimentación recibida de la IAG, indicando qué mejoras podrás incorporar gracias a ella (si las hubiera) y reflexionando brevemente sobre su impacto en tu aprendizaje.
- La declaración de autoría y uso de IAG, conforme a lo indicado en el siguiente apartado.

Puedes aprovechar las observaciones de la IAG para mejorar el código o tus respuestas en la práctica siguiente.

Nota: Es obligatorio entregar el informe en PDF utilizando la plantilla que se facilita en el campus virtual de la asignatura y completar correctamente todos sus apartados; cualquier entrega que no se ajuste a esta plantilla —por ejemplo, subir solo el código o un PDF que no se corresponda con la plantilla— se considerará no válida y conllevará el suspenso de toda la práctica guiada (las cinco entregas).

6. Uso de la IAG, declaración de autoría y buenas prácticas

El uso de herramientas de IAG está permitido en esta práctica únicamente para obtener retroalimentación y sugerencias de mejora sobre el trabajo realizado (por ejemplo, evaluación de código, propuestas de corrección o ayuda en la redacción final). No se permite la elaboración inicial de código, respuestas o informes previos mediante IAG. Todo el desarrollo y resolución de la práctica debe ser realizado personalmente por el estudiante.

Para asegurar la transparencia, la responsabilidad y el uso adecuado de la IAG, la plantilla para la elaboración del informe incluye un apartado final de “Declaración de autoría y uso de IAG”. Mediante esta declaración, el estudiante manifiesta que el trabajo presentado ha sido realizado por él/ella, que asume la responsabilidad sobre su contenido y sobre las decisiones adoptadas durante su elaboración, y que la información proporcionada en el informe acerca del uso de herramientas de Inteligencia Artificial Generativa es completa y veraz.

La entrega del informe supone la aceptación expresa de esta declaración y la asunción de responsabilidad sobre el trabajo presentado.

Recuerda: El uso responsable de la IAG contribuye al aprendizaje autónomo y a la mejora continua, pero no puede sustituir al desarrollo personal de competencias técnicas y críticas. El falseamiento de la autoría será considerado una mala praxis académica.